

Gaussian Priors for Bayesian PDE Inverse Problems: Discretization, Costs, and Two Model Problems

Working Notes

March 2026

Contents

1	Abstract Setup	1
2	Three Specifications of the Covariance	1
2.1	Approach 1: Karhunen–Loève / Spectral	1
2.2	Approach 2: Covariance Kernel	2
2.2.1	A sufficient condition on K for trace class	2
2.3	Approach 3: SPDE / Precision Operator	3
3	From Continuum to Discrete: Costs of Sampling	3
3.1	Discretization of the Kernel Operator	3
3.1.1	Finite element bases: a brief tutorial	3
3.1.2	The mass matrix	4
3.1.3	FEM discretization of the covariance operator	5
3.2	Factorization and Sampling Costs	5
3.3	When to Prefer Eigendecomposition over Cholesky	6
3.4	Concrete Numbers	7
4	SPDE Approach in Detail	7
4.1	Continuum formulation	7
4.2	FEM discretization	8
4.3	The case $\alpha = 1$ (i.e., $\nu = 1 - d/2$, only valid for $d = 1$)	8
4.4	The case $\alpha = 2$ ($\nu = 2 - d/2$; e.g., $\nu = 3/2$ in $d = 1$, $\nu = 1$ in $d = 2$)	9
4.5	Higher α (integer case)	9
4.6	Why this is cheaper: a direct comparison	10
5	Compatibility Principle	10
6	Model Problem I: Bathymetry from the Shallow Water Equations	11
6.1	Setup	11
6.2	Forward Map Numerics	11
6.3	Prior Costs and Compatibility	12

7	Model Problem II: Disorder in Majorana Nanowires	12
7.1	Setup	12
7.2	Forward Map Numerics	12
7.3	Prior Costs and Compatibility	13
8	Comparative Summary	14
9	Practical Recommendations	14

1 Abstract Setup

Let $D \subset \mathbb{R}^d$ be a bounded domain. We seek to infer an unknown function $u : D \rightarrow \mathbb{R}$ from noisy observations of a forward map $\mathcal{G}(u)$ governed by a PDE. We place a Gaussian prior

$$\mu_0 = \mathcal{N}(m, C)$$

on u , where $m \in L^2(D)$ is a mean function and $C : L^2(D) \rightarrow L^2(D)$ is a symmetric, positive, *trace class* operator. Trace class means

$$\text{tr}(C) = \sum_{k=1}^{\infty} \lambda_k < \infty,$$

where $\{\lambda_k\}$ are the eigenvalues of C . This is the necessary and sufficient condition for μ_0 to be a well-defined Gaussian measure on $L^2(D)$ (equivalently, draws have finite expected L^2 norm: $\mathbb{E}_{\mu_0}[\|u - m\|^2] = \text{tr}(C) < \infty$).

There are three natural ways to specify C .

2 Three Specifications of the Covariance

2.1 Approach 1: Karhunen–Loève / Spectral

Choose an orthonormal basis $\{e_k\}_{k \geq 1}$ of $L^2(D)$ (e.g., Fourier modes on a periodic domain) and prescribe eigenvalues $\{\mu_k\}$ directly:

$$Cu = \sum_{k=1}^{\infty} \mu_k \langle e_k, u \rangle e_k, \quad \mu_k > 0, \quad \sum_k \mu_k < \infty.$$

A standard choice is $\mu_k \sim |k|^{-2s}$ for a regularity parameter $s > d/2$; draws then lie almost surely in $H^t(D)$ for any $t < s - d/2$.

Sampling. A draw from $\mathcal{N}(m, C)$ is

$$u = m + \sum_{k=1}^K \sqrt{\mu_k} \xi_k e_k, \quad \xi_k \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1),$$

truncated at level K . Cost: $O(K)$ per sample.

Limitation. Requires an explicit eigenbasis for C that is compatible with the domain geometry and boundary conditions. Essentially limited to periodic domains (Fourier), rectangles (tensor-product sinusoidal bases), or domains where separation of variables applies.

2.2 Approach 2: Covariance Kernel

Specify a symmetric positive-definite kernel $K : D \times D \rightarrow [0, \infty)$ and define

$$Cu(x) = \int_D K(x, y) u(y) dy.$$

This is well-defined and *Hilbert–Schmidt* provided

$$\int_D \int_D |K(x, y)|^2 dx dy < \infty, \quad (1)$$

which guarantees $\sum_k \lambda_k^2 < \infty$. However, **Hilbert–Schmidt does not imply trace class**: trace class requires the strictly stronger condition $\sum_k \lambda_k < \infty$.

2.2.1 A sufficient condition on K for trace class

The most useful sufficient condition is **continuity of K on the diagonal**: if K is continuous on $\overline{D} \times \overline{D}$, then the trace of C is given by

$$\text{tr}(C) = \int_D K(x, x) dx. \quad (2)$$

This is Mercer’s theorem applied to trace: for a continuous, symmetric, positive-definite kernel on a compact domain, the eigenvalues $\{\lambda_k\}$ are non-negative, the eigenfunction expansion $K(x, y) = \sum_k \lambda_k e_k(x) e_k(y)$ converges uniformly, and integrating along the diagonal gives (2). Therefore:

Sufficient condition for trace class. If $K : \overline{D} \times \overline{D} \rightarrow \mathbb{R}$ is continuous, symmetric, and positive definite, then C is trace class with $\text{tr}(C) = \int_D K(x, x) dx < \infty$.

This is the condition to check in practice. For any kernel where $K(x, x)$ is bounded (which includes all stationary kernels $K(x, y) = \varphi(x - y)$ with $\varphi(0) < \infty$), continuity of K on $\overline{D} \times \overline{D}$ immediately gives trace class.

Example 1 (Matérn kernel). *The Matérn kernel with smoothness $\nu > 0$, length scale $\ell > 0$, and variance σ^2 has $K(x, x) = \sigma^2$ for all x . It is continuous on $\overline{D} \times \overline{D}$ for any bounded D . Therefore $\text{tr}(C) = \sigma^2 |D|$ and C is trace class for any $\nu > 0$ on bounded domains. (On $\mathbb{R}^d$ the story is different— $|D| = \infty$ and trace class fails; one needs $\nu > d/2$ for draws to lie in L^2_{loc} .)*

Example 2 (Squared exponential). $K(x, y) = \sigma^2 \exp(-\|x - y\|^2 / (2\ell^2))$. Again $K(x, x) = \sigma^2$ and K is C^∞ , so C is trace class on any bounded D .

Remark 1 (Why Hilbert–Schmidt is not enough). *The Hilbert–Schmidt condition (1) gives $\sum_k \lambda_k^2 < \infty$, which means $\lambda_k \rightarrow 0$ but does not force $\sum_k \lambda_k < \infty$. As a concrete counterexample, take eigenvalues $\lambda_k = 1/k$: then $\sum_k \lambda_k^2 = \pi^2/6 < \infty$ (Hilbert–Schmidt) but $\sum_k \lambda_k = \infty$ (not trace class). The corresponding integral operator would be Hilbert–Schmidt but not trace class, and the associated Gaussian measure would not be well-defined on $L^2(D)$. Mercer’s theorem (continuity of K) rules out this slow decay.*

2.3 Approach 3: SPDE / Precision Operator

The key structural result connecting Approaches 1 and 2 is the **Whittle–Matérn SPDE characterization** [9, 5]: a Matérn covariance with smoothness ν and length scale ℓ corresponds to the Green’s function of the differential operator $(\kappa^2 - \Delta)^\alpha$ where $\alpha = \nu + d/2$ and $\kappa = \sqrt{2\nu}/\ell$. That is,

$$C = \sigma^2(\kappa^2 - \Delta)^{-\alpha},$$

and draws from the prior satisfy the SPDE

$$(\kappa^2 - \Delta)^{\alpha/2}(u - m) = \sigma \mathcal{W},$$

where $\mathcal{W}$ is spatial white noise. On a periodic domain with Fourier modes e_k , this gives eigenvalues $\mu_k = \sigma^2(\kappa^2 + |k|^2)^{-\alpha}$, recovering Approach 1 as a special case.

Advantage. Instead of working with C (a dense integral operator), we work with $C^{-1} = \sigma^{-2}(\kappa^2 - \Delta)^\alpha$ (a differential operator), which is *sparse* when discretized by finite elements. This is the basis for scalable computation on general domains. Section 4 develops this in full detail.

3 From Continuum to Discrete: Costs of Sampling

3.1 Discretization of the Kernel Operator

Given a kernel K at the continuum level, the discrete covariance matrix $\mathbf{C}_N$ depends on how we represent u .

Finite differences. Represent u by nodal values $\mathbf{u} = (u(x_1), \dots, u(x_N))^\top$ on a grid with spacing h . The integral operator is approximated by quadrature:

$$Cu(x_i) \approx \sum_{j=1}^N K(x_i, x_j) u(x_j) h^d,$$

so the discrete covariance matrix acting on coefficient vectors is

$$\mathbf{C}_N = h^d \mathbf{K}, \quad \mathbf{K}_{ij} = K(x_i, x_j).$$

The factor h^d (the quadrature weight) ensures dimensional consistency; omitting it introduces a volume-dependent error that grows/shrinks with refinement.

3.1.1 Finite element bases: a brief tutorial

The finite element method represents u in a space $V_h \subset H^1(D)$ spanned by basis functions $\{\phi_1, \dots, \phi_N\}$ associated with a mesh (triangulation in 2D, partition into intervals in 1D). We write

$$u_h(x) = \sum_{j=1}^N u_j \phi_j(x),$$

where $\mathbf{u} = (u_1, \dots, u_N)^\top$ is the coefficient vector.

1D piecewise-linear (“hat”) functions. Let $D = [a, b]$ with mesh nodes $a = x_0 < x_1 < \dots < x_N < x_{N+1} = b$. The hat function ϕ_j for interior node x_j is defined by

$$\phi_j(x) = \begin{cases} \frac{x - x_{j-1}}{x_j - x_{j-1}} & \text{if } x \in [x_{j-1}, x_j], \\ \frac{x_{j+1} - x}{x_{j+1} - x_j} & \text{if } x \in [x_j, x_{j+1}], \\ 0 & \text{otherwise.} \end{cases}$$

Each ϕ_j is a “tent” centered at x_j , equal to 1 at x_j , zero at all other nodes, and linear in between. The key properties:

- $\phi_j(x_i) = \delta_{ij}$ (the *nodal interpolation* property), so $u_j = u_h(x_j)$ —the coefficients *are* the pointwise values at the nodes.
- Each ϕ_j has compact support (only the two intervals adjacent to x_j), so inner products $\int_D \phi_i \phi_j dx$ are nonzero only when $|i - j| \leq 1$. This makes the resulting matrices *sparse* (tridiagonal in 1D, banded in higher dimensions).
- $\phi_j \in H^1(D)$ (continuous, piecewise smooth), so the FEM space V_h is a subspace of $H^1(D)$.

2D piecewise-linear functions. Triangulate D into triangles. Each interior vertex x_j gets a basis function ϕ_j that equals 1 at x_j , 0 at all other vertices, and is linear on each triangle containing x_j . The support of ϕ_j is the “star” of triangles sharing vertex x_j . Inner products $\int_D \phi_i \phi_j dx$ are nonzero only when x_i and x_j share a triangle edge, giving a sparse matrix with $O(1)$ nonzeros per row (typically 5–7 in 2D).

Higher-order elements. One can use piecewise-quadratic, piecewise-cubic, etc. These add nodes at edge midpoints or interior points and give higher approximation order, but the same sparsity structure. For our purposes, piecewise-linear is sufficient to explain the key ideas.

3.1.2 The mass matrix

The **mass matrix** $\mathbf{M}$ encodes the $L^2(D)$ inner product in the FEM basis:

$$\mathbf{M}_{ij} = \int_D \phi_i(x) \phi_j(x) dx = \langle \phi_i, \phi_j \rangle_{L^2(D)}. \quad (3)$$

This matrix is symmetric positive definite, sparse (same sparsity pattern as the basis—tridiagonal in 1D, banded in 2D), and plays the role of a “change of measure” between the coefficient vector $\mathbf{u}$ and the L^2 function u_h .

Why is $\mathbf{M}$ needed? In the spectral/KL setting, the basis $\{e_k\}$ is orthonormal, so $\langle e_i, e_j \rangle = \delta_{ij}$ and the L^2 inner product of two functions $\langle u, v \rangle = \sum_k \hat{u}_k \hat{v}_k$ is just the Euclidean dot product of coefficient vectors. FEM basis functions are *not* orthonormal: $\langle \phi_i, \phi_j \rangle = \mathbf{M}_{ij} \neq \delta_{ij}$. Consequently, the L^2 inner product of two FEM functions is

$$\langle u_h, v_h \rangle_{L^2} = \mathbf{u}^\top \mathbf{M} \mathbf{v},$$

and $\mathbf{M}$ appears everywhere that the L^2 structure is involved.

Concrete example (1D, uniform mesh). On $[0, 1]$ with N interior nodes and spacing $h = 1/(N + 1)$, the mass matrix for piecewise-linear elements is the tridiagonal matrix

$$\mathbf{M} = \frac{h}{6} \begin{pmatrix} 4 & 1 & & \\ 1 & 4 & 1 & \\ & \ddots & \ddots & \ddots \\ & & 1 & 4 \end{pmatrix}.$$

This is sometimes called the “consistent mass matrix.” A common simplification is the **lumped mass matrix** $\mathbf{M}_{\text{lump}} = h \mathbf{I}$, obtained by summing each row of $\mathbf{M}$ and placing the result on the diagonal. Lumping replaces $\mathbf{M}$ by a scalar multiple of the identity, which effectively reduces FEM to finite differences for the purpose of inner products.

3.1.3 FEM discretization of the covariance operator

The Galerkin projection of the integral operator $Cu(x) = \int_D K(x, y) u(y) dy$ onto the FEM space V_h yields the discrete covariance matrix

$$(\mathbf{C}_N)_{ij} = \int_D \int_D \phi_i(x) K(x, y) \phi_j(y) dx dy. \quad (4)$$

This is a double integral, and its relationship to the pointwise kernel matrix $\mathbf{K}_{ij} = K(x_i, x_j)$ and the mass matrix $\mathbf{M}$ can be understood as follows.

Exact relationship. Think of $\mathbf{C}_N$ as testing the operator C against the basis on both sides:

$$(\mathbf{C}_N)_{ij} = \langle \phi_i, C\phi_j \rangle_{L^2} = \int_D \phi_i(x) \left[\int_D K(x, y) \phi_j(y) dy \right] dx.$$

If the kernel K were exactly a delta function $K(x, y) = \delta(x - y)$ (the identity operator), we would get $\mathbf{C}_N = \mathbf{M}$. For a general smooth kernel, we can use the nodal interpolation property $\phi_j(x_k) = \delta_{jk}$ to write the approximation

$$\int_D K(x, y) \phi_j(y) dy \approx \sum_{\ell} K(x, x_{\ell}) \underbrace{\int_D \phi_{\ell}(y) \phi_j(y) dy}_{\mathbf{M}_{\ell j}},$$

which, after further testing against ϕ_i on the left, gives the rough approximation

$$\mathbf{C}_N \approx \mathbf{M} \mathbf{K} \mathbf{M}, \quad (5)$$

where $\mathbf{K}_{ij} = K(x_i, x_j)$ and $\mathbf{M}$ is the mass matrix. This is *not* exact—it is a quadrature approximation—but it captures the essential structure: the mass matrix “sandwiches” the pointwise kernel evaluation, converting between FEM coefficients and L^2 function values.

With mass lumping. If we replace $\mathbf{M}$ by the lumped mass matrix $\mathbf{M}_{\text{lump}} = h^d \mathbf{I}$ (in d dimensions), then (5) becomes $\mathbf{C}_N \approx h^{2d} \mathbf{K}$, which is (up to factors of h^d) the same as the finite difference discretization. This makes precise the sense in which FD and FEM discretizations of the covariance agree when mass lumping is used.

Spectral. Represent $u = \sum_{k=1}^N \hat{u}_k e_k$. Then $\mathbf{C}_N = \text{diag}(\mu_1, \dots, \mu_N)$.

3.2 Factorization and Sampling Costs

Given an $N \times N$ SPD matrix $\mathbf{C}_N$, sampling from $\mathcal{N}(\mathbf{m}, \mathbf{C}_N)$ requires a square-root factorization $\mathbf{C}_N = \mathbf{A} \mathbf{A}^{\top}$, after which each draw is

$$\mathbf{u} = \mathbf{m} + \mathbf{A} \boldsymbol{\xi}, \quad \boldsymbol{\xi} \sim \mathcal{N}(0, \mathbf{I}_N).$$

This works because $\text{Cov}(\mathbf{A} \boldsymbol{\xi}) = \mathbf{A} \mathbf{I} \mathbf{A}^{\top} = \mathbf{C}_N$. The matrix $\mathbf{A}$ is the “square root” of $\mathbf{C}_N$; different factorizations give different choices of $\mathbf{A}$.

Method	Square root $\mathbf{A}$	Factorization cost	Per-sample	Storage
Cholesky: $\mathbf{C}_N = \mathbf{L}\mathbf{L}^\top$	$\mathbf{A} = \mathbf{L}$ (lower triangular)	$\frac{1}{3}N^3$	$\frac{1}{2}N^2$	$\frac{1}{2}N^2$
Eigen: $\mathbf{C}_N = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$	$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}^{1/2}$	$\sim 9N^3$	$\sim N^2$	$N^2 + N$
Truncated eigen (rank K)	$\mathbf{A} = \mathbf{U}_K\mathbf{\Lambda}_K^{1/2}$ ($N \times K$)	$O(KN^2)$	$O(KN)$	$KN + K$
Spectral/diagonal	$\mathbf{A} = \text{diag}(\sqrt{\mu_1}, \dots, \sqrt{\mu_N})$	0	$O(N)$	N
Sparse Cholesky (SPDE)	$\mathbf{A} = \sigma \mathbf{L}_{\text{sparse}}^{-\top}$ (see §4)	$O(N^{3/2})$ [2D]	$O(N \log N)$ [2D]	$O(N \log N)$

In each case, the per-sample operation is a matrix-vector product $\mathbf{A}\boldsymbol{\xi}$:

- **Cholesky:** $\mathbf{A} = \mathbf{L}$ is lower triangular, so $\mathbf{A}\boldsymbol{\xi}$ is a triangular matrix-vector product (“forward substitution”), costing $\frac{1}{2}N^2$ flops.
- **Eigendecomposition:** $\mathbf{A}\boldsymbol{\xi} = \mathbf{U}(\mathbf{\Lambda}^{1/2}\boldsymbol{\xi})$. First scale $\boldsymbol{\xi}$ by $\sqrt{\lambda_k}$ componentwise ($O(N)$), then multiply by $\mathbf{U}$ (N^2 flops for a dense orthogonal matrix times a vector). Total: $\sim N^2$.
- **Truncated eigen:** $\mathbf{A}\boldsymbol{\xi} = \mathbf{U}_K(\mathbf{\Lambda}_K^{1/2}\boldsymbol{\xi})$ where $\boldsymbol{\xi} \in \mathbb{R}^K$ and $\mathbf{U}_K$ is $N \times K$. The matvec costs $O(KN)$, which is much cheaper than N^2 when $K \ll N$.
- **Spectral:** $\mathbf{A}$ is diagonal, so $\mathbf{A}\boldsymbol{\xi}$ is just componentwise multiplication: $O(N)$.
- **SPDE:** sampling requires a sparse triangular solve rather than a matvec; see Section 4.

3.3 When to Prefer Eigendecomposition over Cholesky

Cholesky is cheaper by a factor of ~ 27 in the factorization and ~ 2 per sample, so for pure sampling it is the default. However, there are real reasons to prefer eigendecomposition:

1. **Truncation / low-rank approximation.** If the eigenvalues decay rapidly (as they do for smooth kernels), keeping only $K \ll N$ dominant modes gives $\mathbf{C}_N \approx \mathbf{U}_K\mathbf{\Lambda}_K\mathbf{U}_K^\top$. Sampling then costs $O(KN)$ instead of $O(N^2)$. For a squared exponential kernel with $N = 10^4$ and $K = 100$, this is a $100\times$ speedup per sample. Cholesky cannot exploit this low-rank structure.
2. **Hyperparameter updates in hierarchical models.** If the kernel depends on hyperparameters $\theta = (\ell, \sigma^2, \nu, \dots)$ and these are updated during MCMC, the Cholesky must be recomputed from scratch at cost $\frac{1}{3}N^3$ for each new θ . With the eigendecomposition, if the eigenvectors are approximately stable (as they often are when only σ^2 or ℓ change moderately), one can update eigenvalues cheaply. More importantly, the eigendecomposition gives the log-determinant as $\log \det \mathbf{C}_N = \sum_k \log \lambda_k$ (sum of N log-eigenvalues, cost $O(N)$), which is needed for the marginal likelihood in model comparison. (Cholesky also gives this: $\log \det \mathbf{C}_N = 2 \sum_i \log L_{ii}$, at $O(N)$ once $\mathbf{L}$ is computed. So the log-determinant itself is not a differentiator; rather, it is the ability to *selectively truncate* the spectral representation.)
3. **Diagnostics and effective dimension.** The eigenvalues reveal the effective dimension of the prior: $d_{\text{eff}} = \sum_k \lambda_k / \lambda_1$ or similar measures. Understanding spectral decay helps calibrate the number of MCMC samples needed and diagnose whether the prior is over- or under-regularizing relative to the data.

4. **Preconditioning the posterior.** In some formulations (e.g., likelihood-informed subspace methods [3]), one projects the posterior onto the leading eigenspace of the prior-preconditioned Hessian. This requires the prior eigenvectors, not just the Cholesky factor.

Remark 2. *If none of these apply and you just need to sample, use Cholesky. It is simpler, faster, and numerically stabler (no risk of negative eigenvalues from roundoff in an ill-conditioned $\mathbf{C}_N$).*

3.4 Concrete Numbers

Assume dense $\mathbf{C}_N$, double precision (8 bytes per entry), single-core at 10^{10} flops/sec.

Grid	N	Storage for $\mathbf{C}_N$	Cholesky time
1D, 1000 pts	10^3	8 MB	< 0.1 sec
2D, 100×100	10^4	800 MB	~ 30 sec
2D, 200×200	4×10^4	12.8 GB	~ 2000 sec
2D, 500×500	2.5×10^5	500 GB	infeasible
3D, $50 \times 50 \times 50$	1.25×10^5	125 GB	infeasible

For the SPDE/sparse approach, the sparse Cholesky of the precision matrix has fill-in $O(N \log N)$ in 2D and $O(N^{4/3})$ in 3D, making it feasible well into the $N \sim 10^5$ – 10^6 range.

4 SPDE Approach in Detail

This section expands on Approach 3 (Section 2.3), filling in the notation, the FEM discretization, and the reason the resulting linear algebra is cheap.

4.1 Continuum formulation

Fix parameters: smoothness $\nu > 0$, length scale $\ell > 0$, marginal variance $\sigma^2 > 0$. Define

$$\kappa = \frac{\sqrt{2\nu}}{\ell}, \quad \alpha = \nu + \frac{d}{2}.$$

The parameter σ^2 is the *marginal variance*—i.e., for a stationary Matérn field on $\mathbb{R}^d$, $\text{Var}(u(x)) = \sigma^2$ for every x . It is related to the SPDE driving noise intensity τ^2 by the normalizing constant

$$\sigma^2 = \frac{\Gamma(\nu)}{\Gamma(\nu + d/2) (4\pi)^{d/2} \kappa^{2\nu}} \cdot \tau^2, \quad (6)$$

where τ^2 is the variance parameter appearing in the SPDE formulation below. (In many references, the SPDE is written in terms of τ rather than σ ; equation (6) converts between them.)

The covariance operator is

$$C = \tau^2 L^{-\alpha}, \quad L := \kappa^2 - \Delta,$$

and the precision (inverse covariance) operator is

$$C^{-1} = \tau^{-2} L^\alpha = \tau^{-2} (\kappa^2 - \Delta)^\alpha.$$

A draw $u \sim \mathcal{N}(m, C)$ satisfies the SPDE

$$L^{\alpha/2}(u - m) = \tau \mathcal{W}, \quad (7)$$

where $\mathcal{W}$ denotes spatial white noise on D . The exponent $\alpha/2$ appears because $L^{\alpha/2}$ is the “square root” of the precision L^α : if we let $w = L^{\alpha/2}(u - m)$, then w has covariance $L^{\alpha/2} \cdot C \cdot L^{\alpha/2} = \tau^2 \cdot L^{\alpha/2} L^{-\alpha} L^{\alpha/2} = \tau^2 I$, which is (scaled) white noise.

What is white noise? Formally, $\mathcal{W}$ is a generalized Gaussian random field with $\mathbb{E}[\mathcal{W}(x)] = 0$ and $\text{Cov}(\mathcal{W}(x), \mathcal{W}(y)) = \delta(x - y)$. It does not live in $L^2(D)$ (its “variance” is $\delta(0) = \infty$), but $L^{-\alpha/2}\mathcal{W}$ does, provided $\alpha > d/2$ (i.e., $\nu > 0$). The SPDE (7) is the rigorous way to define “apply the smoothing operator $L^{-\alpha/2}$ to white noise.”

4.2 FEM discretization

We discretize $L = \kappa^2 - \Delta$ using piecewise-linear finite elements on a mesh with N nodes. Define:

Stiffness matrix:

$$\mathbf{G}_{ij} = \int_D \nabla \phi_i(x) \cdot \nabla \phi_j(x) dx. \quad (8)$$

This is the FEM discretization of $-\Delta$. It is symmetric positive *semidefinite* (with nullspace = constants, if no Dirichlet BCs). Sparse: same sparsity pattern as $\mathbf{M}$.

Mass matrix: $\mathbf{M}_{ij} = \int_D \phi_i(x) \phi_j(x) dx$ as in (3).

Discrete operator: The FEM discretization of $L = \kappa^2 - \Delta$ is the matrix

$$\mathbf{L} = \kappa^2 \mathbf{M} + \mathbf{G}. \quad (9)$$

This is sparse (same sparsity as $\mathbf{M}$ and $\mathbf{G}$), symmetric, and positive definite (since $\kappa^2 > 0$).

Now we need to discretize the SPDE (7). The key step is handling white noise. Testing the SPDE against basis functions ϕ_i :

$$\int_D \phi_i(x) L^{\alpha/2}(u - m)(x) dx = \tau \int_D \phi_i(x) \mathcal{W}(x) dx.$$

The right-hand side $W_i := \int_D \phi_i(x) \mathcal{W}(x) dx$ is a Gaussian random variable with

$$\text{Cov}(W_i, W_j) = \int_D \phi_i(x) \phi_j(x) dx = \mathbf{M}_{ij},$$

so the discrete white noise vector satisfies $\mathbf{W} \sim \mathcal{N}(0, \mathbf{M})$.

4.3 The case $\alpha = 1$ (i.e., $\nu = 1 - d/2$, only valid for $d = 1$)

When $\alpha = 1$ (so $\alpha/2 = 1/2$ and $L^{1/2}$ appears), the discretization is awkward because $L^{1/2}$ is not a local operator. We skip this case.

4.4 The case $\alpha = 2$ ($\nu = 2 - d/2$; e.g., $\nu = 3/2$ in $d = 1$, $\nu = 1$ in $d = 2$)

This is the most important case in practice. The precision operator is $C^{-1} = \tau^{-2}L^2$, and the SPDE is $L(u - m) = \tau \mathcal{W}$ (taking the square root: $L^{\alpha/2} = L^1 = L$). Testing against ϕ_i :

$$\int_D \phi_i \cdot L(u - m) dx = \tau W_i.$$

Using integration by parts on the $-\Delta$ part of L :

$$\kappa^2 \int_D \phi_i (u - m) dx + \int_D \nabla \phi_i \cdot \nabla (u - m) dx = \tau W_i,$$

i.e., $(\kappa^2 \mathbf{M} + \mathbf{G})(\mathbf{u} - \mathbf{m}) = \tau \mathbf{W}$, or simply

$$\mathbf{L}(\mathbf{u} - \mathbf{m}) = \tau \mathbf{W}, \quad \mathbf{W} \sim \mathcal{N}(0, \mathbf{M}). \quad (10)$$

To sample: generate $\mathbf{W} \sim \mathcal{N}(0, \mathbf{M})$, then solve $\mathbf{L} \mathbf{z} = \tau \mathbf{W}$ for $\mathbf{z}$, then set $\mathbf{u} = \mathbf{m} + \mathbf{z}$.

Generating $\mathbf{W} \sim \mathcal{N}(0, \mathbf{M})$. We need the square root of $\mathbf{M}$. Since $\mathbf{M}$ is SPD and sparse, there are two options:

- **Sparse Cholesky:** $\mathbf{M} = \mathbf{L}_M \mathbf{L}_M^\top$, then $\mathbf{W} = \mathbf{L}_M \boldsymbol{\xi}$ with $\boldsymbol{\xi} \sim \mathcal{N}(0, \mathbf{I})$. Cost: one sparse triangular matvec.
- **Mass lumping:** replace $\mathbf{M}$ by $\mathbf{M}_{\text{lump}} = \text{diag}(m_1, \dots, m_N)$ where $m_i = \sum_j \mathbf{M}_{ij}$. Then $\mathbf{W} = \text{diag}(\sqrt{m_i}) \boldsymbol{\xi}$, which is $O(N)$. This introduces a small approximation error but is standard practice [5].

Solving $\mathbf{L} \mathbf{z} = \tau \mathbf{W}$. The matrix $\mathbf{L} = \kappa^2 \mathbf{M} + \mathbf{G}$ is sparse and SPD. Precompute the sparse Cholesky factorization $\mathbf{L} = \mathbf{L}_{\text{sp}} \mathbf{L}_{\text{sp}}^\top$ once; then each sample requires two sparse triangular solves (forward and back substitution). The cost of the sparse Cholesky depends on the mesh structure:

- **1D:** $\mathbf{L}$ is tridiagonal. Cholesky is $O(N)$. Each solve is $O(N)$.
- **2D:** With a good node ordering (nested dissection), the sparse Cholesky has $O(N^{3/2})$ flops for the factorization and $O(N \log N)$ flops per solve, with $O(N \log N)$ storage for the sparse factor.
- **3D:** $O(N^2)$ factorization, $O(N^{4/3})$ per solve.

What is $\mathbf{A}$ for the SPDE approach? Combining the two steps, a sample is $\mathbf{u} = \mathbf{m} + \tau \mathbf{L}^{-1} \mathbf{L}_M \boldsymbol{\xi}$. So the “square root” of the discrete covariance is

$$\mathbf{A} = \tau \mathbf{L}^{-1} \mathbf{L}_M,$$

and $\mathbf{C}_N = \mathbf{A} \mathbf{A}^\top = \tau^2 \mathbf{L}^{-1} \mathbf{M} \mathbf{L}^{-\top}$. Equivalently, the discrete *precision* matrix is

$$\mathbf{Q} := \mathbf{C}_N^{-1} = \tau^{-2} \mathbf{L}^\top \mathbf{M}^{-1} \mathbf{L} = \tau^{-2} \mathbf{L} \mathbf{M}^{-1} \mathbf{L}, \quad (11)$$

where the last equality uses symmetry of $\mathbf{L}$.

4.5 Higher α (integer case)

For $\alpha = 2k$ (even integer), L^k is a local differential operator of order $2k$, and the SPDE $L^k(u - m) = \tau \mathcal{W}$ can be discretized as $\mathbf{L}^k(\mathbf{u} - \mathbf{m}) = \tau \mathbf{W}$. Sampling requires k applications of $\mathbf{L}^{-1}$ to a mass-matrix-colored noise vector. The sparse Cholesky of $\mathbf{L}$ is computed once and reused.

For odd α or non-integer α , one cannot simply iterate $\mathbf{L}^{-1}$. The rational approximation approach of Bolin and Kirchner [1] writes

$$L^{-\alpha} \approx \sum_{j=1}^{n_{\text{rat}}} w_j (L + s_j I)^{-1},$$

where the weights w_j and shifts s_j are chosen to approximate the function $t \mapsto t^{-\alpha}$ on the spectrum of L . Each term $(L + s_j I)^{-1}$ is a sparse solve (same sparsity as $\mathbf{L}$ but shifted), and n_{rat} is typically 5–15.

4.6 Why this is cheaper: a direct comparison

Suppose we want to sample from $\mathcal{N}(0, C)$ with a Matérn covariance ($\nu = 1$, $d = 2$, so $\alpha = 2$) on a triangulated domain with N nodes.

	Dense kernel approach	SPDE approach
Setup	Form $\mathbf{K}$ (N^2 entries), Cholesky ($\frac{1}{3}N^3$)	Assemble $\mathbf{L}, \mathbf{M}$ (sparse, $O(N)$ entries), sparse Cholesky of $\mathbf{L}$ ($O(N^{3/2})$ in 2D)
Storage	$\frac{1}{2}N^2$ (dense triangle)	$O(N \log N)$ (sparse factor)
Per sample	$\frac{1}{2}N^2$ (dense triangular matvec)	$O(N \log N)$ (sparse triangular solve)
$N = 10^4$ setup	3.3×10^{11} flops (~ 30 sec)	$\sim 10^6$ flops (< 0.001 sec)
$N = 10^4$ per sample	5×10^7 (~ 0.005 sec)	$\sim 10^5$ (< 0.001 sec)
$N = 10^4$ storage	400 MB	~ 1 MB
$N = 10^5$	Infeasible (40 TB storage)	Setup $\sim 10^{7.5}$ flops, sample $\sim 10^6$
$N = 10^6$	Infeasible	Setup $\sim 10^9$ flops, sample $\sim 10^7$

The fundamental reason the SPDE approach wins is that it *never forms the dense covariance matrix*. It works entirely with the sparse precision operator C^{-1} , and sampling from C reduces to solving a sparse linear system against C^{-1} .

5 Compatibility Principle

The prior discretization must be compatible with the forward map discretization. Specifically, the numerical representation of u used for sampling from $\mathcal{N}(m, C)$ should match (or be cheaply convertible to) the representation consumed by the PDE solver computing $\mathcal{G}(u)$. Mismatches introduce:

- An interpolation cost at every likelihood evaluation (potentially $O(N^2)$ or worse).
- Discretization artifacts: interpolation errors that are invisible to the prior but distort the likelihood, polluting the posterior.
- Inconsistent mesh refinement behavior, breaking discretization-invariance properties of algorithms like pCN.

The natural pairings are:

Prior	Forward solver	Compatibility
Spectral/KL (Fourier)	Pseudospectral / spectral Galerkin	Exact: same basis
SPDE/FEM	FEM on the same mesh	Exact: same nodal values
Dense kernel + FD grid	FD on the same grid	Exact: same grid
Kernel on grid A	FEM on mesh B	Interpolation needed

6 Model Problem I: Bathymetry from the Shallow Water Equations

6.1 Setup

The unknown is the bottom topography $b : D \rightarrow \mathbb{R}$ where $D \subset \mathbb{R}^d$ ($d = 1$ or $d = 2$) with periodic boundary conditions. The forward map $\mathcal{G}(b)$ solves the shallow water equations

$$\partial_t h + \nabla \cdot (h\mathbf{v}) = 0, \quad \partial_t(h\mathbf{v}) + \nabla \cdot (h\mathbf{v} \otimes \mathbf{v} + \tfrac{1}{2}gh^2\mathbf{I}) = -gh\nabla b,$$

where h is the water depth, $\mathbf{v}$ the velocity, and g the gravitational constant. Observations are (noisy) measurements of the surface elevation $\eta = h + b$ and/or velocity at sensor locations.

6.2 Forward Map Numerics

1D, periodic. A pseudospectral (Fourier collocation) solver is natural. The state variables h, v are represented spectrally with N modes; nonlinear products are computed in physical space via FFT. Cost per time step: $O(N \log N)$. For $T/\Delta t$ time steps with CFL-limited $\Delta t \sim h = 1/N$, total cost per forward solve: $O(N^2 \log N)$.

2D, periodic. Same pseudospectral approach on an $N_x \times N_y$ grid ($N = N_x N_y$ total DOFs). Cost per time step: $O(N \log N)$ via 2D FFT. Total: $O(N \cdot (N_x + N_y) \log N)$ accounting for the CFL condition $\Delta t \sim \min(h_x, h_y)$.

Alternative: Finite volume / discontinuous Galerkin on unstructured meshes. More flexible for non-periodic BCs, but $O(N)$ per time step with larger constants.

6.3 Prior Costs and Compatibility

	Prior sample	Forward solve	Compatible?	Verdict
<i>1D periodic, N modes/points</i>				
Spectral/KL	$O(N)$	$O(N^2 \log N)$	Exact	Ideal
Dense kernel	$O(N^2)$ setup; $O(N^2)$ /sample	$O(N^2 \log N)$	Grid match	Feasible for $N \leq 10^3$
SPDE/FEM	$O(N)$	$O(N^2 \log N)$	Interpolation	Unnecessary overhead
<i>2D periodic, $N = N_x \times N_y$ total DOFs</i>				
Spectral/KL	$O(N)$	$O(N^{3/2} \log N)$	Exact	Ideal
Dense kernel	$O(N^3)$ setup	$O(N^{3/2} \log N)$	Grid match	Infeasible for $N > 10^4$
SPDE/FEM	$O(N \log N)$	$O(N^{3/2} \log N)$	Interpolation	Viable if non-periodic

Bottom line for bathymetry (periodic). The spectral/KL prior is the clear winner. The periodic domain and pseudospectral forward solver are perfectly matched. Sampling cost is negligible relative to the forward solve in both 1D and 2D. There is no reason to use the kernel or SPDE approach unless one moves to non-periodic geometries (e.g., a coastal domain with open boundaries), at which point the SPDE/FEM prior paired with a DG or FV solver becomes the natural choice.

7 Model Problem II: Disorder in Majorana Nanowires

7.1 Setup

The unknown is a disorder potential $V : [0, L] \rightarrow \mathbb{R}$ along a 1D nanowire of length L . The forward map $\mathcal{G}(V)$ returns the differential conductance $G(E)$ at specified energies E , computed as follows:

1. **Tight-binding BdG Hamiltonian.** Discretize $[0, L]$ into N_{sites} lattice sites. The Bogoliubov-de Gennes Hamiltonian $H_{\text{BdG}}(V)$ is a $4N_{\text{sites}} \times 4N_{\text{sites}}$ sparse (block-tridiagonal) matrix encoding hopping, spin-orbit coupling, Zeeman splitting, superconducting pairing, and the on-site disorder V .
2. **Scattering matrix.** Attach semi-infinite leads and compute the scattering matrix $S(E)$ at energy E . This requires solving a linear system of size $4N_{\text{sites}} \times 4N_{\text{sites}}$ (effectively a discrete Sturm-Liouville/Green's function problem for the block-tridiagonal H_{BdG}).
3. **Conductance.** Extract the conductance from the transmission block of $S(E)$ via the Landauer-Büttiker formula: $G(E) = (e^2/h) \text{tr}(t^\dagger t)$.

7.2 Forward Map Numerics

The dominant cost is the scattering matrix computation. Two standard approaches:

Direct (recursive Green's function). Exploit the block-tridiagonal structure. Sweep left-to-right, inverting one 4×4 block at each site. Cost: $O(N_{\text{sites}} \cdot 4^3) = O(64 N_{\text{sites}})$ per energy E . For

N_E energy values, total cost is $O(64 N_{\text{sites}} \cdot N_E)$. Typical scales: $N_{\text{sites}} \sim 10^2\text{--}10^3$, $N_E \sim 10^1\text{--}10^2$, so $\sim 10^5\text{--}10^7$ flops per forward evaluation. This is **very cheap**.

Transfer matrix. Multiply 4×4 transfer matrices along the chain. Same $O(N_{\text{sites}})$ scaling but numerically unstable for long wires; stabilized variants exist.

The key structural feature: the forward map takes a **vector** $\mathbf{V} = (V_1, \dots, V_{N_{\text{sites}}})$ of on-site potentials. The unknown function $V(x)$ is sampled at lattice sites—this is a finite difference discretization with $N = N_{\text{sites}}$ and spacing a (the lattice constant).

7.3 Prior Costs and Compatibility

Since the physical model is defined on a discrete lattice, $N = N_{\text{sites}}$ is small (typically $10^2\text{--}10^3$). This completely changes the cost picture.

	Prior sample	Forward solve	Compatible?	Verdict
<i>1D lattice, $N = N_{\text{sites}} \sim 10^2\text{--}10^3$</i>				
Spectral/KL	$O(N)$	$O(N)$	Need DFT	Excellent
Dense kernel + Cholesky	$\frac{1}{3}N^3$ setup; $\frac{1}{2}N^2$ /sample	$O(N)$	Exact	Excellent
SPDE/FEM	$O(N)$ (tridiag)	$O(N)$	Exact	Excellent

Detailed cost comparison for $N = 500$, $N_E = 50$:

Operation	Flops	Time @ 10^{10} flops/s	Notes
Forward solve	$\sim 1.6 \times 10^6$	< 0.001 sec	$64 \times 500 \times 50$
Spectral prior sample	~ 500	negligible	$O(N)$
Dense kernel sample	$\sim 1.25 \times 10^5$	negligible	$\frac{1}{2}N^2$
Cholesky factorization (once)	$\sim 4.2 \times 10^7$	~ 0.004 sec	$\frac{1}{3}N^3$
Eigendecomposition (once)	$\sim 1.1 \times 10^9$	~ 0.1 sec	$\sim 9N^3$

Bottom line for the nanowire problem. At these scales ($N \sim 10^2\text{--}10^3$), *all three prior approaches are trivially affordable*. The forward solve itself is cheap ($\sim 10^6$ flops), and even the dense kernel Cholesky is a one-time cost under a second. This means you have **complete freedom in prior choice**—the numerical bottleneck is not the prior but rather the total number of MCMC samples needed for posterior exploration. The dense kernel approach becomes attractive here precisely because it offers maximum modeling flexibility (arbitrary covariance structure, nonstationary kernels, learned hyperparameters) at negligible computational overhead.

Remark 3 (Spectral/KL for the lattice model). *On a 1D lattice with N sites, a natural choice is the DFT basis. If the forward map takes pointwise values $V_j = V(x_j)$, converting from spectral coefficients $\hat{V}_k$ to site values requires an FFT: $O(N \log N)$. At $N = 500$ this is ~ 4500 flops, still negligible. However, the lattice is not periodic (the wire has open ends attached to leads), so the Fourier basis is not the most natural; a DCT basis (cosine modes) or direct dense kernel may be more appropriate.*

8 Comparative Summary

	Bathymetry (SWE)	Nanowire (BdG)
Domain	$D \subset \mathbb{R}^d$, $d = 1, 2$; periodic	$[0, L]$, discrete lattice; open ends
N (DOFs)	10^2 – 10^5	10^2 – 10^3
Forward cost	$O(N^2 \log N)$ [1D]; $O(N^{3/2} \log N)$ [2D]	$O(N)$ per energy
Bottleneck	Forward solve dominates	Neither; total MCMC iterations
Best prior (periodic)	Spectral/KL	Any; dense kernel most flexible
Kernel approach viable?	1D yes; 2D infeasible (dense)	Yes, easily
SPDE approach needed?	Only for non-periodic domains	No advantage at this scale
Key compatibility	Spectral prior $\leftrightarrow$ pseudospectral PDE	Site values $\leftrightarrow$ on-site potential

The two problems occupy opposite ends of the computational spectrum. For bathymetry in 2D, the forward solve is expensive and the prior *must* be cheap and compatible (spectral is the only practical choice on periodic domains; SPDE/FEM is needed for general geometries). For the nanowire, the forward map is so inexpensive that prior cost is irrelevant—one should choose the prior purely on modeling grounds (e.g., smoothness, stationarity, ability to encode physical constraints), without any numerical constraint.

9 Practical Recommendations

- For the bathymetry problem on periodic domains:** Use the spectral/KL prior with eigenvalues $\mu_k \sim (\kappa^2 + |k|^2)^{-\alpha}$ (Matérn-like decay) and a pseudospectral SWE solver. The pCN / mpCN / MTpCN proposals are trivially cheap. If non-periodic domains are needed in the future, transition to an SPDE/FEM prior paired with a DG forward solver.
- For the nanowire problem:** Start with the dense kernel approach. Choose a Matérn or squared-exponential kernel $K(x_i, x_j)$ with hyperparameters (length scale ℓ , variance σ^2 , smoothness ν). Precompute the Cholesky factorization once (cost $\sim \frac{1}{3}N^3 \sim 10^7$ flops for $N = 500$, negligible). This gives maximum flexibility for:
 - Hierarchical inference on ℓ, σ^2, ν (update the Cholesky when hyperparameters change—still cheap at $N \leq 10^3$).
 - Nonstationary priors if physical considerations suggest the disorder varies differently near the contacts vs. the bulk.
 - Model comparison via evidence (the log-determinant $\log \det \mathbf{C}_N$ is a byproduct of the Cholesky: $\log \det \mathbf{C}_N = 2 \sum_i \log L_{ii}$).
- For pCN/multiproposal methods in general:** The key requirement is the ability to draw $\xi \sim \mathcal{N}(0, C)$ quickly. All three approaches satisfy this when properly matched to the problem

scale and domain geometry. The spectral approach gives $O(N)$ draws with zero setup on simple geometries; the SPDE approach gives $O(N \log N)$ draws on complex geometries; the dense kernel gives $O(N^2)$ draws (after $O(N^3)$ setup) but is limited to $N \leq 10^3$ – 10^4 .

References

- [1] D. Bolin and K. Kirchner, *The rational SPDE approach for Gaussian random fields with general smoothness*, J. Comput. Graph. Statist. **29** (2020), 274–285.
- [2] S. L. Cotter, G. O. Roberts, A. M. Stuart, and D. White, *MCMC methods for functions: modifying old algorithms to make them faster*, Statist. Sci. **28** (2013), 424–446.
- [3] T. Cui, K. J. H. Law, and Y. M. Marzouk, *Likelihood-informed dimension reduction for non-linear inverse problems*, Inverse Problems **30** (2014), 114015.
- [4] M. Dashti and A. M. Stuart, *The Bayesian approach to inverse problems*, in: Handbook of Uncertainty Quantification, Springer, 2017, pp. 311–428.
- [5] F. Lindgren, H. Rue, and J. Lindström, *An explicit link between Gaussian fields and Gaussian Markov random fields: the stochastic partial differential equation approach*, J. R. Statist. Soc. B **73** (2011), 423–498.
- [6] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006.
- [7] H. Rue and L. Held, *Gaussian Markov Random Fields: Theory and Applications*, Chapman & Hall/CRC, 2005.
- [8] A. M. Stuart, *Inverse problems: a Bayesian perspective*, Acta Numerica **19** (2010), 451–559.
- [9] P. Whittle, *Stochastic processes in several dimensions*, Bull. Int. Statist. Inst. **40** (1963), 974–994.